Customer Support LLM Chatbot Project_v2

December 18, 2025

Project Overview In this project, I fine-tuned a lightweight large language model (Llama-3.2-1B) to act as a customer support chatbot. The model was trained using the Bitext Customer Support LLM Chatbot Training Dataset, which contains realistic customer–agent conversations across common support scenarios such as order issues, refunds, account problems, and delivery delays.

The goal of the project was to adapt a general-purpose language model to respond with the tone, structure, and behaviour expected from a professional customer support agent. To achieve this, I used supervised fine-tuning (SFT) with LoRA adapters, focusing on improving response clarity, empathy, instruction-following, and consistency, while keeping the model efficient enough to run on consumer-grade hardware.

The final model demonstrates how a small, fine-tuned LLM can deliver practical, domain-specific performance for customer support use cases without the cost or complexity of training large models from scratch.

connect with me on LinkedIn: https://www.linkedin.com/in/stephenelufisan/

[1]:
```
from datasets import load_dataset, Dataset
import torch

ds = load_dataset(
    'bitext/Bitext-customer-support-llm-chatbot-training-dataset',
    split = "train"
)

ds = ds[:]

ds = Dataset.from_dict(ds)
```

[2]:
```
def merge_example(row):
    row['conversation'] = f"Query: {row['instruction']}\nResponse:␣
 ↪{row['response']}"
    return row

data = ds.map(merge_example)

print(data[0]['conversation'])
```

```
Map:   0%|          | 0/26872 [00:00<?, ? examples/s]
```

```
Query: question about cancelling order {{Order Number}}
Response: I've understood you have a question regarding canceling order {{Order
Number}}, and I'm here to provide you with the information you need. Please go
ahead and ask your question, and I'll do my best to assist you.
```

[3]:
```python
from transformers import AutoTokenizer
from transformers import AutoModelForCausalLM

model_name= "meta-llama/Llama-3.2-1B"

model = AutoModelForCausalLM.from_pretrained(model_name, torch_dtype=torch.
 ↪bfloat16, device_map="cuda",)
tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token
```

```
`torch_dtype` is deprecated! Use `dtype` instead!
W1218 09:11:50.160000 24480 site-
packages\torch\distributed\elastic\multiprocessing\redirects.py:29] NOTE:
Redirects are currently not supported in Windows or MacOs.
```

[4]:
```python
torch.backends.cuda.matmul.allow_tf32 = True
torch.backends.cudnn.allow_tf32 = True
```

```
c:\Users\eeluf\miniconda3\envs\ml\Lib\site-
packages\torch\backends\__init__.py:46: UserWarning: Please use the new API
settings to control TF32 behavior, such as
torch.backends.cudnn.conv.fp32_precision = 'tf32' or
torch.backends.cuda.matmul.fp32_precision = 'ieee'. Old settings, e.g,
torch.backends.cuda.matmul.allow_tf32 = True, torch.backends.cudnn.allow_tf32 =
True, allowTF32CuDNN() and allowTF32CuBLAS() will be deprecated after Pytorch
2.9. Please see
https://pytorch.org/docs/main/notes/cuda.html#tensorfloat-32-tf32-on-ampere-and-
later-devices (Triggered internally at C:\actions-
runner\_work\pytorch\pytorch\pytorch\aten\src\ATen\Context.cpp:85.)
  self.setter(val)
```

[ ]:
```python
from trl import SFTTrainer, SFTConfig
from peft import LoraConfig

lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=["q_proj", "v_proj"]
)
```

```python
sft_config = SFTConfig(
    output_dir="model_v1/meta-llama",
    dataset_text_field="conversation",

    # speed/efficiency
    max_length=1024,
    packing=False,

    per_device_train_batch_size=8,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,

    bf16=True,
    fp16=False,

    optim="paged_adamw_8bit",
    gradient_checkpointing=True,
    gradient_checkpointing_kwargs={"use_reentrant": False},

    dataloader_num_workers=4,
    logging_steps=10,
    save_steps=500,
)

sft_trainer = SFTTrainer(
    model=model,
    processing_class=tokenizer,
    train_dataset=data,
    args=sft_config,
    peft_config=lora_config,
)

sft_trainer.train()
```

```
WARNING:tensorflow:From c:\Users\eeluf\miniconda3\envs\ml\Lib\site-
packages\tf_keras\src\losses.py:2976: The name
tf.losses.sparse_softmax_cross_entropy is deprecated. Please use
tf.compat.v1.losses.sparse_softmax_cross_entropy instead.


Adding EOS to train dataset:   0%|          | 0/26872 [00:00<?, ? examples/s]

Tokenizing train dataset:   0%|          | 0/26872 [00:00<?, ? examples/s]

Truncating train dataset:   0%|          | 0/26872 [00:00<?, ? examples/s]

The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and
generation config. The model config and generation config were aligned
accordingly, being updated with the tokenizer's values. Updated tokens:
{'pad_token_id': 128001}.
```

```
<IPython.core.display.HTML object>
```

[ ]:
```
TrainOutput(global_step=2520, training_loss=0.8272877857798622,
metrics={'train_runtime': 30681.8761, 'train_samples_per_second': 2.627,
'train_steps_per_second': 0.082, 'total_flos': 1.2049665993385574e+17,
'train_loss': 0.8272877857798622, 'epoch': 3.0})
```

[7]:
```
from peft import get_peft_model

llama_model = get_peft_model(model, lora_config)
llama_model.print_trainable_parameters()
```

```
trainable params: 1,703,936 || all params: 1,237,518,336 || trainable%: 0.1377

c:\Users\eeluf\miniconda3\envs\ml\Lib\site-packages\peft\mapping_func.py:72:
UserWarning: You are trying to modify a model with PEFT for a second time. If
you want to reload the model with a different config, make sure to call
`.unload()` before.
  warnings.warn(
c:\Users\eeluf\miniconda3\envs\ml\Lib\site-
packages\peft\tuners\tuners_utils.py:282: UserWarning: Already found a
`peft_config` attribute in the model. This will lead to having multiple adapters
in the model. Make sure to know what you are doing!
  warnings.warn(
```

connect with me on LinkedIn: https://www.linkedin.com/in/stephenelufisan/

[8]:
```
from transformers import pipeline, AutoTokenizer, AutoModelForCausalLM
import torch

model_id = "model_v1/meta-llama/checkpoint-1500"

tokenizer = AutoTokenizer.from_pretrained(model_id)

model = AutoModelForCausalLM.from_pretrained(
    model_id,
    torch_dtype=torch.bfloat16,
    device_map="cuda"
)

pipe = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
)

prompt = (
    "Query: This is the third time I've contacted you and no one is helping me!
 ↪\n"
```

```
    "Response:"
)

outputs = pipe(
    prompt,
    max_new_tokens=120,
    do_sample=True,
    temperature=0.7,
    top_p=0.9,
    repetition_penalty=1.1,
    eos_token_id=tokenizer.eos_token_id,
)

generated_text = outputs[0]["generated_text"]
response = generated_text[len(prompt):].strip()

print(response)
```

```
Device set to use cuda

Thank you for bringing this to our attention. We understand that your
frustration has escalated, and we apologize for any inconvenience caused. Your
patience and persistence are greatly appreciated. To better assist you, could
you please provide us with more details about the issue you encountered during
your previous attempts to contact us? Your insights will help us identify the
root cause of the problem and find a suitable solution. We value your feedback
as it enables us to continuously improve our services. Thank you again for
bringing this to our attention, and rest assured, we're committed to resolving
this matter promptly and efficiently. How can we further
```

```
[ ]:
```